Systèmes de fichiers et arborescence Linux

Partie 1 : Les systèmes de fichiers
Partie 2 : La structure des répertoires (FHS)

FORMATEUR
Yohanh Galilée SENGA

Sommaire

1.1	Rôle et fonctionnement d'un système de fichiers	Partie 1
1.2	Notions clés : partition, bloc, inode, journal	Partie 1
1.3	Liens physiques et liens symboliques	Partie 1
1.4	Les principaux systèmes de fichiers	Partie 1
1.5	Montage et gestion des systèmes de fichiers	Partie 1
2.1	Principes de l'arborescence Linux	Partie 2
2.2	Le standard FHS : vue d'ensemble	Partie 2
2.3	Description détaillée des répertoires	Partie 2
2.4	Types de fichiers et permissions	Partie 2
2.5	Commandes de navigation et de gestion	Partie 2

PARTIE 1

Les systèmes de fichiers

1.1 Rôle et fonctionnement d'un système de fichiers

Un **système de fichiers** (*File System*, abrégé FS) est la méthode utilisée par le système d'exploitation pour organiser, stocker, retrouver et protéger les données sur un support (SSD, disque dur, clé USB, partition réseau). Sans système de fichiers, un disque ne serait qu'une suite d'octets bruts, sans notion de fichier ni de dossier.

- **Organiser :** structurer les données en fichiers et répertoires, sous forme d'arborescence.
- **Allouer :** décider quels blocs du disque contiennent quel fichier et gérer l'espace libre.
- **Décrire :** conserver les métadonnées (taille, dates, propriétaire, permissions).
- **Protéger :** contrôler les droits d'accès et préserver la cohérence des données en cas de panne.
- **Accélérer :** utiliser des caches et des structures d'index pour retrouver rapidement l'information.

Le chemin d'une requête sous Linux

Sous Linux, toutes les applications passent par une couche d'abstraction commune, le **VFS** (*Virtual File System*). Elle offre la même interface (open, read, write, close) quel que soit le système de fichiers réellement utilisé : c'est ce qui permet de lire indifféremment une partition ext4, une clé USB en FAT32 ou un partage réseau.

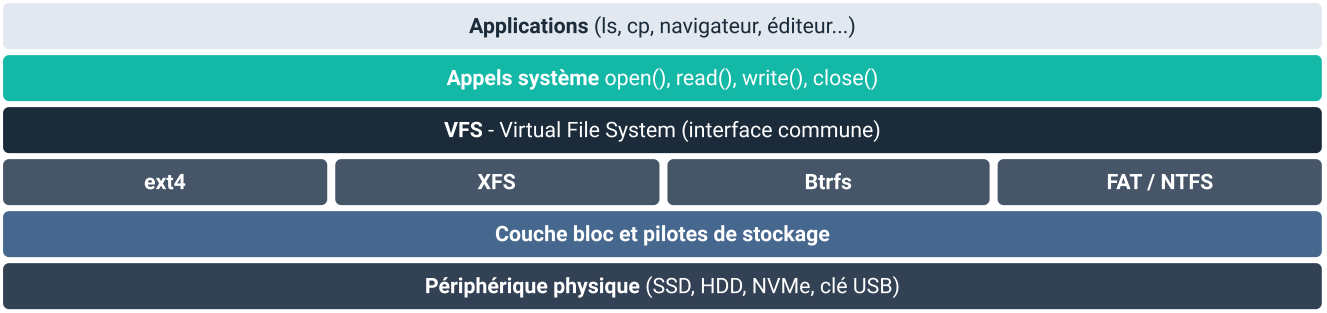


Figure 1 - Couches traversées lors d'un accès à un fichier

1.2 Notions clés

Notion	Définition
Disque / périphérique bloc	Support de stockage vu par Linux comme un fichier spécial dans /dev (ex. : `/dev/sda`, `/dev/nvme0n1`).
Partition	Portion d'un disque délimitée par une table de partitions (MBR ou GPT). Chaque partition peut recevoir son propre système de fichiers (ex. : `/dev/sda1`).
Formatage	Création d'un système de fichiers sur une partition (commande `mkfs`). Il initialise les structures de gestion : superbloc, table d'inodes, zones de données.
Bloc	Plus petite unité d'allocation du disque (souvent 4 Ko). Un fichier occupe un nombre entier de blocs.

Superbloc	Zone qui décrit le système de fichiers lui-même : type, taille, nombre de blocs et d'inodes, état (propre ou non).
Inode	Structure qui contient toutes les informations d'un fichier, **sauf son nom** et son contenu.
Journal	Zone où le FS note les opérations avant de les appliquer, pour récupérer rapidement un état cohérent après un arrêt brutal.
Point de montage	Répertoire de l'arborescence auquel on rattache un système de fichiers pour le rendre accessible.

L'inode : la carte d'identité d'un fichier

Sur les systèmes de fichiers de type Unix, chaque fichier est représenté par un **inode** identifié par un numéro unique dans le système de fichiers. Le nom du fichier n'est pas stocké dans l'inode : il est enregistré dans le **répertoire** qui contient le fichier, sous forme d'une association « nom, numéro d'inode ».

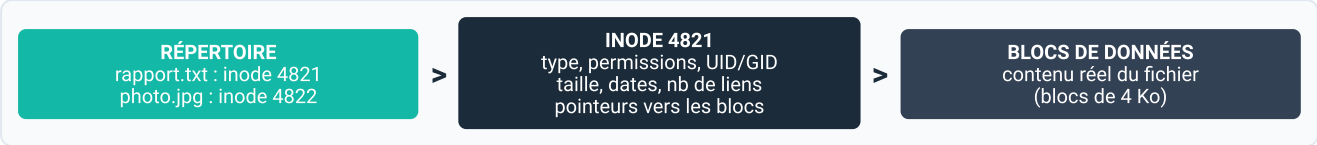


Figure 2 - Du nom du fichier à ses données : répertoire, inode, blocs

Information stockée dans l'inode	Exemple
Type de fichier	Fichier ordinaire, répertoire, lien symbolique, périphérique...
Permissions	rwxr-xr-x (lecture, écriture, exécution)
Propriétaire et groupe	UID et GID
Taille	En octets
Horodatages	atime (accès), mtime (modification), ctime (changement inode)
Nombre de liens physiques	Nombre de noms pointant vers cet inode
Pointeurs vers les blocs	Adresses des blocs contenant les données

Commande utile : `ls -i` affiche les numéros d'inode, et `stat fichier` détaille toutes les métadonnées. `df -i` indique l'utilisation des inodes : un système de fichiers peut être « plein » parce qu'il n'a plus d'inodes disponibles, même s'il reste de la place.

La journalisation

Lors d'une coupure de courant ou d'un plantage, une écriture à moitié terminée peut laisser le système de fichiers incohérent. Les FS modernes (ext4, XFS, Btrfs, NTFS) utilisent un **journal** :

1. Journal	Le FS écrit d'abord dans le journal la description de l'opération à réaliser.
2. Application	Il exécute ensuite réellement l'opération sur le disque.
3. Validation	Il marque l'opération comme terminée dans le journal.
Après un crash	Au redémarrage, le système rejoue ou annule les opérations incomplètes du journal : la vérification est quasi instantanée.

1.3 Liens physiques et liens symboliques

Linux permet de donner plusieurs noms ou raccourcis à un même fichier grâce aux liens, créés avec la commande `ln`.

Critère	Lien physique (hard link)	Lien symbolique (symlink)
Commande	`ln source lien`	`ln -s source lien`
Principe	Un second nom pointant vers le **même inode**.	Un fichier spécial contenant le **chemin** vers la cible.
Suppression original	Les données restent accessibles tant qu'il reste un lien.	Le lien devient « cassé » (pointe dans le vide).
Entre FS	Impossible (même FS uniquement).	Possible.
Sur répertoire	Interdit pour les utilisateurs.	Possible.
Usage typique	Sauvegardes économes en espace.	Raccourcis, versions de bibliothèques.

1.4 Les principaux systèmes de fichiers

Système	Caractéristiques principales	Usage typique
ext4	Évolution d'ext3. Journalisé, très stable et éprouvé, bonnes performances générales.	Système par défaut (Debian, Ubuntu). Postes et serveurs.
XFS	Journalisé, conçu pour les très gros volumes et les fichiers volumineux. Ne peut pas être réduit.	Par défaut sur RHEL. Serveurs, bases de données.
Btrfs	Copy-on-write, instantanés (snapshots), sommes de contrôle, RAID intégré.	Fedora, openSUSE. Sauvegardes.
ZFS	Gestionnaire de volumes et FS combinés, intégrité très poussée, snapshots.	Serveurs de stockage (NAS), virtualisation.
F2FS	Conçu pour la mémoire flash (SSD, eMMC, cartes SD).	Smartphones, systèmes embarqués.
FAT32 / exFAT	Simples et universels. FAT32 limite à 4 Go ; exFAT supprime la limite. Pas de droits Unix.	Clés USB, cartes SD, échange inter-OS.
NTFS	FS de Windows, journalisé. Lu et écrit sous Linux via ntfs3 ou ntfs-3g.	Disques partagés avec Windows.
tmpfs / procfs	FS virtuels : tmpfs vit en RAM ; /proc et /sys exposent le noyau.	/tmp, /run, /proc, /sys.
NFS / SMB	FS réseau : accès à des fichiers distants comme s'ils étaient locaux.	Partages d'entreprise.

> **Comment choisir ?**
> Pour un usage général, **ext4** est le choix sûr. **XFS** convient aux gros volumes de données, **Btrfs** ou **ZFS** si vous avez besoin de snapshots et de contrôle d'intégrité, **exFAT** pour échanger des fichiers avec Windows et macOS.

1.5 Montage et gestion des systèmes de fichiers

Contrairement à Windows qui attribue une lettre (C:, D:) à chaque volume, Linux n'a qu'**une seule arborescence**. Pour accéder au contenu d'une partition, il faut la **monter** sur un répertoire existant (point de montage).

Exemple de configuration d'un système

Périphérique	Point de montage	Type	Contenu
`/dev/nvme0n1p1`	`/boot/efi`	vfat	Partition de démarrage UEFI
`/dev/nvme0n1p2`	`/`	ext4	Système (racine)
`/dev/nvme0n1p3`	`/home`	ext4	Données des utilisateurs
`/dev/sdb1`	`/mnt/data`	xfs	Disque de données supplémentaire
`tmpfs`	`/tmp`	tmpfs	Fichiers temporaires en RAM

Ajouter un nouveau disque : procédure type

1. Identifier	`lsblk`	Lister les disques et partitions détectés.
2. Partitionner	`sudo fdisk /dev/sdb`	Créer une table de partitions et une partition.
3. Formater	`sudo mkfs.ext4 /dev/sdb1`	Créer le système de fichiers sur la partition.
4. Préparer	`sudo mkdir /mnt/data`	Créer le point de montage.
5. Monter	`sudo mount /dev/sdb1 /mnt/data`	Rattacher la partition à l'arborescence.
6. Permanent	Éditer `/etc/fstab`	Monter automatiquement au démarrage.
7. Vérifier	`df -h`	Contrôler l'espace et les montages actifs.

Le fichier /etc/fstab

Ce fichier décrit les systèmes de fichiers à monter au démarrage. Il est recommandé d'utiliser l'**UUID** de la partition plutôt que son nom de périphérique.

```
# peripherique point type options dump fsck UUID=3f2a9c1e-5b7d-4e8a-9a10-0c1d2e3f4a5b / ext4 de
```

Commandes utiles	`blkid` (UUID), `df -h` (espace), `du -sh` (taille dossier), `mount/umount`, `fsck` (vérification)

> **Attention**
> Ne jamais lancer `fsck` sur un système de fichiers monté en lecture/écriture, et vérifier deux fois le nom du périphérique avant un `mkfs`.

PARTIE 2

La structure des répertoires Linux

2.1 Principes de l'arborescence Linux

- **Une arborescence unique :** tout part de la **racine**, notée `/`. Les autres disques y sont rattachés par montage.
- **Tout est fichier :** documents, répertoires, périphériques (`/dev/sda`), processus (`/proc`) sont vus comme des fichiers.
- **Sensible à la casse :** `Rapport.txt` et `rapport.txt` sont deux fichiers distincts.
- **Séparateur `/` :** contrairement au `\` de Windows. Les fichiers commençant par un point (`.bashrc`) sont cachés.
- **Le standard FHS :** le *Filesystem Hierarchy Standard* définit le rôle de chaque répertoire principal.

Notation	Signification	Exemple
`/`	Racine de l'arborescence	`cd /`
Chemin **absolu**	Part de la racine (commence par /)	`/home/yohanh/docs/note.txt`
Chemin **relatif**	Part du répertoire courant	`docs/note.txt`
`.` / `..`	Répertoire courant / Répertoire parent	`./script.sh` / `cd ..`
`~` / `-`	Dossier personnel / Dossier précédent	`cd ~` / `cd -`

2.2 Le standard FHS : vue d'ensemble

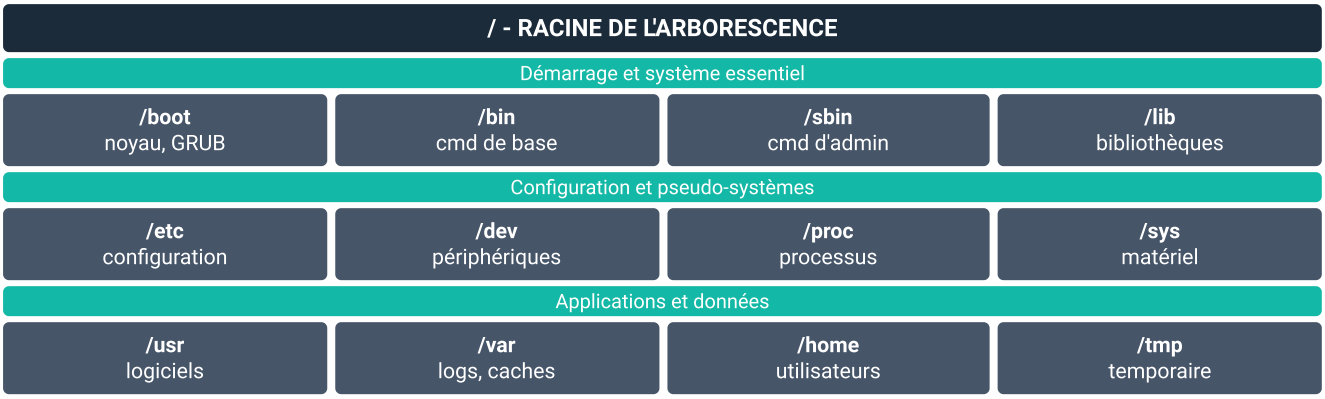


Figure 3 - Les répertoires principaux de la racine, regroupés par fonction

2.3 Description détaillée des répertoires

Répertoire	Rôle	Contenu typique
`/`	Racine de toute l'arborescence.	Tous les répertoires ci-dessous.
`/boot`	Fichiers nécessaires au démarrage du système.	Noyau (`vmlinuz`), GRUB.
`/bin` / `/sbin`	Commandes essentielles (système / administration).	`ls, cp, bash` / `fdisk, mount`
`/lib, /lib64`	Bibliothèques partagées et modules du noyau.	Fichiers `.so` , modules.
`/etc`	Fichiers de **configuration** du système (texte).	`passwd, fstab, hosts, ssh/`
`/dev`	Fichiers spéciaux représentant les périphériques.	`/dev/sda, /dev/null, /dev/tty`
`/proc` / `/sys`	Pseudo-FS : infos processus, noyau et matériel.	`/proc/cpuinfo, /sys/class`
`/home` / `/root`	Répertoires personnels (utilisateurs / root).	`/home/yohanh` , config root.
`/usr`	Logiciels et ressources installés (hiérarchie sec.).	`/usr/bin, /usr/share`
`/var`	Données **variables** (logs, caches, files).	`/var/log, /var/cache`
`/tmp` / `/run`	Fichiers temporaires / état d'exécution en RAM.	Fichiers temporaires, PID.
`/opt` / `/srv`	Logiciels tiers / données des services (web, FTP).	`/opt/google` , `/srv/www`
`/mnt` / `/media`	Montages manuels / automatiques (clés USB).	Disques montés.

> **Fusion /usr (usrmerge)**
> Sur les distributions modernes, `/bin` , `/sbin` et `/lib` sont des liens symboliques vers `/usr/bin` , `/usr/sbin` et `/usr/lib` .

Zoom sur /usr, /var et /etc

Chemin	Rôle
`/usr/bin` / `/usr/sbin`	Programmes utilisateurs et administration non essentiels au boot.
`/usr/share` / `/usr/local`	Données indépendantes de l'architecture / Logiciels installés à la main.
`/var/log` / `/var/lib`	Journaux du système (syslog) / État permanent des bases de données et paquets.
`/etc/passwd` / `/etc/shadow`	Comptes utilisateurs / Mots de passe chiffrés (accès root).
`/etc/fstab` / `/etc/hosts`	Table des montages / Correspondances IP locales.

2.4 Types de fichiers et permissions

La commande `ls -l` affiche le type de fichier (1er caractère) et neuf caractères de permissions :

```
$ ls -l -rw-r--r-- 1 yohanh formation 1240 oct. 19 10:12 rapport.txt drwxr-xr-x 2 yohanh format
```

Symbole	Type	Exemple
`-` / `d` / `l`	Fichier ordinaire / Répertoire / Lien symbolique	`rapport.txt` / `/home`
`c` / `b`	Périphérique caractère / bloc (disque)	`/dev/tty` / `/dev/sda`
`s` / `p`	Socket de communication / Tube nommé (FIFO)	`/run/systemd/...`

Lecture des permissions

Les permissions sont divisées en trois groupes : **propriétaire (u)**, **groupe (g)** et **autres (o)**.



Figure 4 - Décomposition de la chaîne -rwxr-xr-- (octal 754)

Droit	Octal	Sur un fichier	Sur un répertoire
r (read)	4	Lire le contenu	Lister les fichiers
w (write)	2	Modifier le contenu	Créer / supprimer des fichiers
x (execute)	1	Exécuter le programme	Traverser (accéder au dossier)

`chmod 754 fichier`	rwx pour u (7), r-x pour g (5), r-- pour o (4).
`chown user:grp fichier`	Change le propriétaire et le groupe.

2.5 Commandes de navigation et de gestion

Commande	Fonction	Exemple
`pwd`	Affiche le répertoire courant.	`pwd`
`cd`	Change de répertoire.	`cd /var/log`
`ls -lah`	Liste le contenu avec détails et tailles lisibles.	`ls -lah /etc`
`tree -L 2`	Affiche l'arborescence sous forme d'arbre.	`tree -L 2 /`
`mkdir -p`	Crée un répertoire et ses parents si besoin.	`mkdir -p proj/src`
`cp -r` / `mv`	Copie récursive / Déplace ou renomme.	`cp -r doc copie`
`rm -r`	Supprime des fichiers ou dossiers récursivement.	`rm -r dossier`
`ln -s`	Crée un lien symbolique.	`ln -s /etc/hosts h`
`find`	Recherche des fichiers selon des critères.	`find / -name "*.conf"`
`file` / `stat`	Détermine le type réel / Affiche métadonnées et inodes.	`stat rapport.txt`

À retenir

- Un **système de fichiers** organise les données sur un support ; le **VFS** offre une interface commune à tous les systèmes (ext4, XFS, Btrfs, FAT, NTFS...).
- Chaque fichier est décrit par un **inode** ; son nom est conservé dans le répertoire parent.
- La **journalisation** garantit un redémarrage rapide et cohérent après un crash.
- Linux n'a qu'**une seule arborescence** partant de `/` : les partitions sont **montées**, et `/etc/fstab` automatise ce montage.
- Le standard **FHS** fixe le rôle de chaque répertoire (`/etc`, `/var`, `/home`, `/usr`...).
- Les **permissions** (rwx pour u, g, o) protègent l'accès aux fichiers et se gèrent via `chmod` et `chown`.